



ICS3U Resource Book

Revised August 2013

1.2 A First Program

We begin our study of Java with a program that prints a message greeting the world.

Example 1

```
class Greet
{
    // This program prints a simple message
    public static void main (String[] args)
    {
        System.out.println("Hello, world");
    }
}
```

Let us look in detail at the components of the program in Example 1.

1. **class**

All Java programs are contained in a *class*. The start of a class is indicated by the *reserved word* **class**. Java contains about fifty reserved words, all of which are listed later in this chapter.

2. **Greet**

This is the name that identifies the class. The choice of a name is, within limits, up to the programmer but, customarily, classes have names that start with an upper case letter. We chose the name **Greet** because the program greets the world but we could have called it **FirstProgram** if we had chosen to do so. In fact, as far as the computer is concerned, we could have called the class **Athena** or **Saskatoon** but it is better, for the benefit of your human readers, to use a meaningful name.

3. **{ }**

The beginning and end of any section of a Java program are indicated

by *brace brackets*, { and }. The beginning and end of the class are indicated by the { in the second line and the } in the last line of the program.

4. `// This program ...`

The two adjacent slashes indicate a *comment* in a Java program. Anything on a line after `//` is ignored by the compiler. Although they do not affect the way that the program works, comments can be very useful to any person trying to understand a program. A comment can also be indicated in a Java program by using `/*` to start the comment and `*/` to end it. Using this style, we could have written our comment as

```
/* This program prints a simple message */
```

Usually, comments that appear entirely on one line are written using the `//` style while those that occupy multiple lines are written using the `/* ... */` style.

5. `public static void main (String[] args)`

Java programs consist of one or more *methods*, (sometimes called *functions* or *procedures* in other programming languages). If the program is run directly by the Java interpreter, then exactly one of its methods must be called `main`.¹ Our program has only one method so it must be called `main`. This line is the *header* of the `main` method. The other words and symbols in the header will be explained later. For now, just use them, exactly as shown.

6. `{ }`

The inner pair of brace brackets defines the beginning and end of the definition of the `main` method, just as the outer pair of brace brackets defines the limits of the definition of the class.

7. `System.out.println("Hello, world");`

This *statement* calls on a method called `println` to send the *string* of characters `Hello world` contained in double quotes to some output device (probably your computer's screen). Whatever the output device is, we can think of it as a page of paper. The `println` method, given a string of characters, writes the string on the page and then moves on to the next line of the page, ready to print the next item, if there is one. As we suggested in discussing the header of the method,

¹Such programs are sometimes called *application programs*. As we will see later when we study *applets*, Java has other types of programs that do not have a `main` method.

the details of the use of `println` will be made clear later. For now, just use it exactly as shown, with an upper case `S` on `System`, dots between `System`, `out`, and `println`, and including both the parentheses and the terminating semi-colon.

You should notice in Example 1 the way in which the parts of the program have been *indented*. Everything within the brace brackets that enclose the body of the class is indented two spaces. Similarly, everything within the brace brackets that enclose the body of the `main` method is indented two more spaces past the header of the method.

Indentation is not necessary for the computer but it can improve the readability for a human. Since your programs should be clear as well as correct, you should always indent them. Your instructor might suggest an indentation style that is slightly different from the one that we use. The important thing is to use a style that is both clear and consistent.

Example 2

As far as a computer is concerned, the program of the previous example is exactly equivalent to either of the following:

```
class Greet{public static void main(String[]
args){System.out.println("Hello, world");}}
```

or

```
class          Greet          {
    public
static

    main(      String          [      void
               args            )
    {
        System.out.println
        (      "Hello, world"
               )          ;
    }
}
```

Notice the ways in which blanks are used in Example 2.

1. At least one blank must be used to separate two adjacent words. Thus, `classGreet` or `voidmain` would be incorrect.
2. If a symbol appears between two words, it can be used as a separator so that a blank is no longer required. For example, `main(String` and `main ( String` are equivalent.
3. Extra blanks, even entire blank lines, do not usually change the meaning of the surrounding symbols. Blanks can even be inserted before or after dots so that `System . out . println` would be correct. (It is, however, considered bad style to have blanks around dots; you should avoid it.) Finally, extra blanks inside a string (inside double quotes) will *not* be ignored so that the output produced by printing the strings `"Hello , world"` and `"Hello,world"` would be different.

A number of variations can be made to our basic printing technique.

1. To print a blank line, we can simply write
`System.out.println();`
2. If we want to print a string that is longer than we have room for on one line of our program, we *cannot* simply continue the string onto a second line. If we were to do so, the compiler would get upset and refuse to compile our program. Instead, to print a very long string, we can write something like the following:

```
System.out.println("This will print a verrrrrry long"
    + " string on one line by joining"
    + " these strings into one string");
```

The process of joining strings is called *concatenation*. The plus sign used with strings acts as the concatenation operator in Java.

3. If we do not wish to jump to a new line after printing a string, we can use the method `print` instead of `println` as illustrated in the next example.

Example 3

The following program illustrates some uses of `print` and `println`.

```
class PrintDemo
{
    public static void main (String[] args)
    {
        System.out.println("Never put off till tomorrow");
        System.out.println();
        System.out.print("what you can do");
        System.out.println(" the day");
        System.out.println();
        System.out.println("after" + " " + "tomorrow.");
    }
}
```

This will produce the following output.

```
Never put off till tomorrow

what you can do the day

after tomorrow.
```

Since the beginning and end of a string to be written by `print` or `println` are marked by double quotes, how do we print a string that contains double quotes? Java's solution is to use a *backslash* character, `\`, immediately in front of a double quote to signal the computer that it should print the character `"`.

Example 4

The statement

```
System.out.println("Angela said, "
                    + "\"That's ridiculous, Bram.\"");
```

would print

```
Angela said, "That's ridiculous, Bram."
```

The technique used to print a double quote can be used in a number of other ways. For example, to print a backslash, you should write `\\`. To jump to a new line, you should write `\n`. (The `n` indicates a *new* line.) These pairs are called *escape sequences*. A complete list of escape sequences can be found in Appendix C.

Example 5

The statement

```
System.out.println("A backslash: \\na double quote: \");
```

would produce the output

A backslash: \
a double quote: "

Exercises 1.2

1. What does this program print?

```
class Advice
{
    public static void main (String[] args)
    {
        System.out.print("If at first ");
        System.out.println("you don't succeed" + ",");
        System.out.print("failure may be ");
        System.out.println("your style");
    }
}
```

2. Rewrite the following program using the indentation style shown in the text.

```
class Quote{public static void main(String[]args){
System.out.print("The unexamined life");
System.out.println(" is not worth living.");}}
```

3. Write a *single* Java statement that would print the following, exactly as shown.

```
A slash is "/"
while
a backslash is "\"
```

4. The following program contains a number of errors. Rewrite the program with the errors corrected.

```
class BadForm
public void main (string() args);
{
    System.Out.Println('What's wrong with this?')
}
```

5. Write a Java program to print your name in giant letters as shown in the following example.

```
BBBB  EEEEE  A  TTTT  RRRR  III  X  X
B  B  E      A A  T    R  R  I  X  X
B  B  E      A A  T    R  R  I  X  X
BBBB  EEE    AAAAA  T    RRRR  I  X
B  B  E      A A  T    R  R  I  X  X
B  B  E      A A  T    R  R  I  X  X
BBBB  EEEEE  A  A  T    R  R  III  X  X
```

6. Write Java programs to print each design. You might find it helpful to use squared paper in planning the appearance of your output.

<pre>PARALLELOG A R R A ALLELOGRAM</pre>	<pre>APEZO R I TRAPEZOID</pre>	<pre>D I I A A M M O O N N D</pre>
--	-----------------------------------	---------------------------------------

7. Write a complete Java program to print your name and address as they would appear on the envelope of a letter written to you at your home.

If we wish to store an integer value with a larger range than that permitted by the type `byte`, we can use 16 bits. Integer values stored in 16 bits are said to be of type `short`. Their range is from -2^{15} to $2^{15} - 1$.

Java permits two other types of integer values: `int` and `long`. The following table gives a summary of the four integer types.

Type	Size (in bits)	Range	Approximate Range
<code>byte</code>	8	-2^7 to $2^7 - 1$	± 100
<code>short</code>	16	-2^{15} to $2^{15} - 1$	$\pm 30\,000$
<code>int</code>	32	-2^{31} to $2^{31} - 1$	$\pm 2\,000\,000\,000$
<code>long</code>	64	-2^{63} to $2^{63} - 1$	$\pm 9 \times 10^{18}$

The `int` type is adequate for most tasks and you should use it for almost all situations that require an integer. The types `byte` or `short` can be used if we need to save space in memory while `long` can be used if we require integers with an extended range.

Example 1

The integer value 2 000 000 could be represented by an `int` (maximum value $2^{31} - 1 = 2\,147\,483\,647$) but not by a `short` (maximum value $2^{15} - 1 = 32\,767$).

Integer constants in Java are written as a sequence of decimal digits with or without a preceding plus or minus sign. They should not be written with any leading zeros.³ An integer constant is normally stored as an `int` value. If the letter L or l is placed at the end of a constant it is stored as a `long` value.

Example 2

The following are all valid integer constants. The first five would be stored as `int` values while the last one would be stored as a `long` value.

4 0 113 -357462 +23 9876543210L

³In Java, numbers with leading zeros are not considered to be standard, base 10 numerals. For a detailed explanation of this somewhat bizarre rule, see Appendix C.

1.5 Other Primitive Types

In addition to the four integer types discussed in the previous section, Java has four other *primitive types* of values: `float`, `double`, `char`, and `boolean`. We will examine them in this section.

To store numeric values containing decimals or to store integers whose size exceeds the largest value that can be stored in a `long` integer, Java uses two *floating point* types: `float` and `double`. In both of these types, the numbers are represented in a form that is similar to *scientific notation*. In scientific notation, the value 576.34 is written in the form 5.7634×10^2 . In a floating point representation, the number is separated into two parts: the *mantissa* (the digits) and the *exponent*. The next table gives a summary of the two floating point types.

Type	Size (in bits)	Precision	Approximate Range
<code>float</code>	32 1 for sign 8 for exponent 23 for mantissa	at least 6 decimal digits	$\pm 3.4 \times 10^{38}$
<code>double</code>	64 1 for sign 11 for exponent 52 for mantissa	at least 15 decimal digits	$\pm 1.8 \times 10^{308}$

Floating point constants in Java can be written as an optional plus or minus sign, followed by a sequence of one or more digits together with a decimal point. A floating point constant is normally stored as a `double` value. If the letter `F` or `f` is placed at the end of a constant, it is stored as a `float` value. In the past, memory was more costly and operations on `float` values were much faster than those on `double` values. The situation has improved since then and now the type `double` is almost always used for floating point values. Although `double` values can represent a wide range of numbers to a very high precision, you should realize that they cannot represent *every* real number; to do so would require an infinite

number of bits. Instead, during calculations, values are rounded if necessary. Although rounding errors can become critical in some situations, for our purposes, `double` values will be adequately precise.

Example 1

Each of the following are valid floating point constants.

5.23 .3 2818. -0.0002 6.7f

The first four would be stored as `double` values while the last one would be stored as a `float` value.

Floating point constants can also be written in a form similar to that used in scientific notation with one part showing the digits and the other showing the position of the decimal point. The system is illustrated in the next example.

Example 2

(a)	5.6e2	represents the value	5.6×10^2	= 560
(b)	37E-4	represents the value	37×10^{-4}	= 0.0037
(c)	-0.667e-2	represents the value	-0.667×10^{-2}	= -0.00667

Groups of bits can also be used to represent *characters*. Early computers used six bits to represent a character, permitting a total of $2^6 = 64$ different characters. With this scheme, only the 10 digits, 26 upper case letters of the alphabet, and a variety of punctuation marks and other symbols could be represented. The details of the coding scheme varied from computer to computer. More recently, a seven bit encoding called ASCII (American Standard Code for Information Interchange) has gained wide acceptance, allowing the use of both upper and lower case characters and far more special characters.

The creators of Java have chosen a newer representation called *Unicode* that uses 16 bits to represent each character. With 16 bits, a total of $2^{16} = 65\,536$ characters can be represented. With such a large number of character codes available, the characters and symbols used by any of the

world's languages along with a wide variety of symbols and shapes can be represented by a character in Unicode. You need not be concerned about the details of the Unicode encoding scheme but, if you are interested, look at <http://unicode.org/>.

Characters in Java programs are of type `char`. We show constants representing single characters by enclosing them in apostrophes (also known as single quotes).

Example 3

The following are all valid `char` constants.

`'A'` `'$'` `'+'` `'7'`

To represent the apostrophe character, we use an escape sequence, just as we did inside strings (in the previous section). Thus, the constant that represents the single apostrophe can be written in the form `'\''`. Similarly, the newline character (that causes a jump to a new line) can be written as `'\n'`.

It is also possible to represent a character by specifying the bit pattern as a hexadecimal value in a Unicode escape sequence. For example, the character Σ can be specified by writing `\u03a3`. If you are interested in exploring this further, see Appendix C. You do *not* need to know anything about this form of character representation in order to follow any of the material in this text.

The last of Java's eight primitive types is called *boolean*. The boolean type represents a truth value. There are only two boolean values: `true` and `false`. These words, `true` and `false` are reserved by Java for boolean values and cannot be used in any other way. Boolean values are represented using one bit.

Exercises 1.5

1. Rewrite in standard decimal form.

(a) `2.94e1`

(b) `0.0004E3`

- | | |
|-------------------------|---------------------------|
| (c) <code>-2e-3</code> | (d) <code>26.77e-3</code> |
| (e) <code>-54E-3</code> | (f) <code>-.3e1</code> |
2. The `double` constants `6.73` and `67.3e-1` would be stored identically in Java. Find three other representations of each of the following `double` values.
- | | |
|-------------------------|----------------------------|
| (a) <code>27E4</code> | (b) <code>-0.0058e2</code> |
| (c) <code>-87.69</code> | (d) <code>-3e-4</code> |
3. Identify the type of each constant.
- | | |
|-------------------------|------------------------|
| (a) <code>-3.8e2</code> | (b) <code>53L</code> |
| (c) <code>-12</code> | (d) <code>false</code> |
| (e) <code>17.4</code> | (f) <code>2E3</code> |
| (g) <code>.5f</code> | (h) <code>'2'</code> |

1.6 Identifiers and Variables

If we are to create programs that do something other than write messages, we will need some way of storing and retrieving values. The memory cells used to store values that can change during the execution of a program are known as *variables*. In order to manipulate values in Java, we create *identifier names* (or, more simply, *identifiers*) for our variables, the methods that contain them, and the classes that contain the methods. The rules for creating any type of identifier are:

1. Any character used in an identifier must be a letter of the alphabet (either upper case or lower case, in any alphabet supported by Unicode),⁴ a digit (0, 1, ..., 9), the underscore character, or a Unicode currency symbol such as `$`, `£`, or `¥`. Java distinguishes between upper case and lower case letters so that, for example, `x` and `X` are recognized as different characters.

⁴This includes accented Latin letters such as `é`, `ö`, and `č`. It also includes Greek, Cyrillic and other alphabets. Of course, your system might not be able to print such letters but they are valid in Java.

2. The first character cannot be a digit.
3. An identifier that you create must not be one of the following *reserved words*, each of which has a pre-defined and unalterable meaning in Java.

abstract	assert	boolean	break	byte
case	catch	char	class	const
continue	default	do	double	else
enum	extends	false	final	finally
float	for	goto	if	implements
import	instanceof	int	interface	long
native	new	null	package	private
protected	public	return	short	static
strictfp	super	switch	synchronized	this
throw	throws	transient	true	try
void	volatile	while		

In addition to these rules that *must* be followed in creating identifiers, there are also a number of conventions that *should* be used.

1. Currency symbols (such as dollar signs) in identifiers are used by system programs and should be avoided by users.
2. Long variable or method identifiers are written primarily using lower case letters with either upper case letters or underscores used to make the meaning clearer. For example, a variable to represent a starting time could be `timeAtStart` or `time_at_start`. In this text, we will use the combination of upper case and lower case letters, not the underscore character.
3. Class identifiers start with an upper case letter; variable and method identifiers do not. Other characters in class identifiers follow the convention described for variables.

Example 1

From the following list, select (with reasons) those names that should not be used as identifiers of Java variables.

firstChoice	this	second-time
3rdTime	monthly_total	Last
valueIn\$	x27	word Length

Names that should not be used as variable identifiers are:

- `this` – a reserved word
- `second-time` – contains a hyphen
- `3rdTime` – starts with a digit
- `Last` – valid but, by convention, variable identifiers start with a lower case character
- `valueIn$` – valid but, by convention, currency symbols are avoided
- `word Length` – contains a blank

It is good programming practice to use meaningful identifiers in your programs. Although it takes a bit more work to type a name like `highMark` than it does to type `hm`, the former gives the reader a much clearer idea of the use to which the identifier is to be put. In large programs, it is not unusual to have hundreds of different identifiers and, in these programs, meaningful names are almost absolutely essential for reliability. Although you will not likely be writing such large programs for some time, descriptive names will improve the clarity of any program and should be used at all times.

To reserve space in memory for variables, we must write a *declaration* in which we specify the type of a variable and its identifier. A simple declaration has the form

```
<type> <identifier>;
```

Note that the declaration, like any statement in Java, is terminated by a semi-colon.

Example 2

To request that space in memory be allocated to an `int` variable to be called `age`, we could write

```
int age;
```

We can illustrate the result of this declaration as follows:

`age`



The box represents the space in memory allocated to the variable `age`. Notice that, at this point, there is nothing in the box. This is because, at this point, the variable `age` does not yet have a value.

It is not necessary to use a separate declaration for each variable that we want to create. We can request that space be allocated to many variables of the same type in a single declaration using a statement of the form

`<type> <identifier1>, <identifier2>, . . . , <identifierk>;`

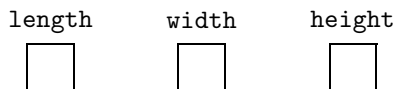
Example 3

The declaration

```
double length, width, height;
```

creates three variables: `length`, `width`, and `height`, all of type `double`.

The result of this declaration is shown in the following diagram.



When a variable is declared in a method, the declaration can appear at the start of the method (just after the opening brace bracket) or it can appear later. A variable must, however, be declared before it is first used. The practice that we follow in this text is (usually) to declare a variable just before we are about to use it.

Exercises 1.6

1. What is the difference between identifiers used for classes and those used for variables and methods?
2. Identify, with reasons, any identifiers that should not be used for Java variables.
 - (a) `digitSum`
 - (b) `switch`
 - (c) `retail price`
 - (d) `heightPlusDepth`
 - (e) `value@Start`
 - (f) `priceIn£`

- (g) `number-of-wins`
- (h) `ageDuGarçon`
- (i) `average_age`
- (j) `This`

3. Rewrite each fragment using the minimum number of declarations.

- (a) `int quantity;`
`double price;`
`int size;`
- (b) `double income;`
`char taxCode;`
`double rate;`
`double discount;`

4. A program is to be written that records the results of a coin-tossing experiment. The program will need variables to record the number of heads, the number of tails, the percentage of heads (correct to four significant digits), and the experimenter's first and last initials. Write declarations to reserve space in memory of the appropriate type for each of these variables. Be sure to use identifiers that show clearly the purpose of each variable.

1.7 Assigning Values to Variables

Declaring variables, as we did in the last section, simply reserves space in the computer's memory and associates that space with the identifier that we have chosen; it does not give a value to that variable. If we were to write a program that attempted to print the value of a variable after it had been declared but before it had been given a value, Java would give us an error message when we attempted to compile the program.

Once variables are declared, there are many ways in which we can give values to them. One of the simplest ways is to use an *assignment statement* of the form

`<identifier> = <value>;`

Although the form of an assignment looks like an equation, it is quite different. The statement causes an action to be performed: the `<value>` on the right side of the equals sign is stored in the location reserved for the `<identifier>`. Note that, as with a declaration, an assignment statement is terminated by a semi-colon.

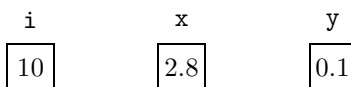
When we first give a value to a variable, we say that we are *initializing* the variable. Initialization of a variable can be done as soon as it has been declared. In fact, a declaration and assignment can be done in one statement. The next example illustrates this.

Example 1

The statements

```
int i = 10;  
double x = 2.8, y = 0.1;
```

will declare the variables `i`, `x`, and `y` and give them values of 10, 2.8, and 0.1 respectively. The results of executing these statements are illustrated in the next diagram.



In any assignment of the form

`<identifier> = <value>;`

the type of `<value>` must be compatible with the type of `<identifier>`. Normally this means that `<identifier>` and `<value>` are of identical types but there are some exceptions.

Generally, Java permits conversions from one type to another if it should be safe to make the conversion. For example, Java permits us to assign a `byte` value to an `int` variable because any value that can be represented by a `byte` variable (of 8 bits) can obviously be represented in an `int` variable (of 32 bits). In Java, such a conversion is referred to as a *widening* conversion because the new representation is wider (has a greater range) than the old representation. The following table summarizes the valid widening conversions permitted in Java.

From	To
byte	short, int, long, float, double
short	int, long, float, double
char	int, long, float, double
int	long, float, double
long	float, double
float	double

Conversions other than the ones listed above can result in the loss of information. Such conversions are called *narrowing conversions* and should be treated with great care. Because data can be lost in narrowing conversions, the Java compiler complains if you attempt to perform such a conversion, even if the value in the wider representation can safely be stored in the narrower representation.

If you are certain that a narrowing conversion is safe, you can force Java to make the conversion by performing a *cast* from one type to another. To perform a cast, write the name of the desired type, in parentheses, in front of the value to be converted.

Example 2

The following fragment is valid. It will cause the value 2.5 to be stored in the `float` variable `f`.

```
float f;
f = (float) 2.5;
```

Java permits the assignment of integer *constants* to either `short` or `byte` variables without using a cast.

Example 3

The following statement is valid. It will cause the integer value 75 to be stored in the `short` variable `s`.

```
short s = 75;
```

Here no cast is necessary because the constant integer value being assigned to the `short` variable is within the permissible range for `short` values and

the compiler can see that the assignment is valid. If we were to write

```
short s = 75000;
```

the compiler would notice that 75 000 is outside the range of a `short` variable and it would produce an error message.

A cast is a kind of promise to the compiler that you know what you are doing. If you lie to the compiler, it will trust you and compile the program, but the result may not be correct.

Example 4

The fragment

```
short s = (short) 75000;  
System.out.println(s);
```

will compile and execute without producing an error message. Unfortunately, when it is executed, the value that it prints will be **9464**!

If a cast is used to convert a floating point type to an integer type, then any fractional part will be lost.

Example 5

The following fragment will store the value 87 in the `int` variable `i` and -32 in `j`.

```
int i = (int) 87.65;  
int j = (int) -32.999;
```

It is also possible to use a cast to convert numerical values to type `char`. For technical reasons beyond the scope of this book, these are always considered to be narrowing conversions, even conversions from `byte` to `char` (where the number of bits is increasing) or from `short` to `char` (where the number of bits is unchanged). On the other hand, as noted in the preceding

table, a `char` can be converted to a variety of numerical types. The result of such a conversion is a non-negative integer value — the value of the bit pattern of the character, if this pattern were considered to represent a binary numeral. If you want to gain a better understanding of this, see Appendix C.

Note that the type `boolean` is not mentioned in the list of widening conversions. This is because variables of type `boolean` cannot be converted to any of the seven other primitive types. It is also true that none of the other primitive types can be converted to the type `boolean`.

The right hand sides of assignment statements do not always consist of constant values. An assignment statement can also be used to copy a value from one variable to another.

Example 6

The following fragment will assign the value 1234 to the `int` variable `i` and the value 2.71828 to the `double` variable `d`. Note that a cast is used to convert the `long` to an `int` but no cast is necessary for the widening conversion from `float` to `double`.

```
long l = 1234L;
float f = 2.71828F;
int i = (int) l;
double d = f;
```

Exercises 1.7

1. Suppose that the following declarations have been made:

```
int number;
double mass;
float density;
char symbol;
byte age;
long limit;
```

State, with reasons, which assignments are illegal.

- (a) `number = 32.0;` (b) `mass = 37;`
 - (c) `density = 1.52e0;` (d) `symbol = +;`
 - (e) `age = 19;` (f) `limit = 10000;`
2. Which of the following conversions always require a cast?
- (a) `int` to `char` (b) `byte` to `int`
 - (c) `double` to `float` (d) `float` to `short`
 - (e) `double` to `long` (f) `int` to `float`
3. Write a program fragment that declares the `short` variable `subTotal`, then declares the `int` variable `grandTotal`, initializing `grandTotal` to zero, and finally copies the value from `grandTotal` into `subTotal`.
4. Find and correct the errors.
- (a) `short s = 25;` (b) `int i = 50;`
 `int i = int(s);` `short s = i;`

1.8 String Variables

You may have heard that Java is an *object oriented* language. Although there are only eight different primitive types, the number of object types is unlimited. Many are predefined but we can also create our own types of objects, to serve our own purposes. We will be studying objects very extensively later on but, for now, we will only be using one of the most common and useful types of objects available in Java — the *string*. Strings in Java are objects of type `String` (written exactly as shown, with an upper case `S`). We have already encountered string constants but we can also have string variables. To declare a variable of type `String`, we proceed as we would for any primitive type.

Example 1

The declaration

```
String s;
```

will create a variable `s` of type `String` whose value is undefined, as shown in the next diagram.



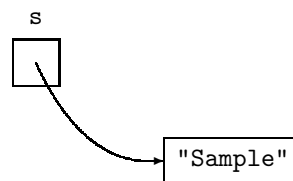
To assign a value to a string variable, we can proceed in a manner similar to that used for assignments to primitive types. The result of such an assignment is, however, quite different from what we have seen with primitive types.

Example 2

If we have declared `s` to be of type `String`, we can assign a value to `s` by writing

```
s = "Sample";
```

Here Java creates a new `String` object containing the string `"Sample"` but this is *not* stored in `s`. Instead, the value in `s` becomes a *reference* to the location in memory that contains the string. The result is illustrated in the next diagram where the arrow indicates that `s` contains a reference to the location occupied by the string.



Before going on, you should be sure that you understand what is happening here. A variable of type `String`, like `s` in our examples, does not

actually contain a string. Instead, it contains a reference to the actual string object which is stored elsewhere in memory. Because `s` contains a reference to a string, we say that it is an example of a *reference type* variable (as opposed to a primitive type variable that actually contains its value).

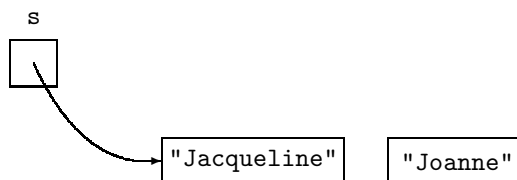
Once we create a string, its value cannot be changed. Because of this, we say that strings are *immutable* objects. This does *not* mean that we cannot change values of variables that refer to strings.

Example 3

The fragment

```
String s = "Joanne";  
s = "Jacqueline";
```

first sets the variable `s` to refer to a string object containing "Joanne" and then sets `s` to refer to a new string object containing "Jacqueline" (at which point the object containing "Joanne" is lost). The result is illustrated in the next diagram.



Although the string containing "Joanne" still exists, it is no longer accessible.

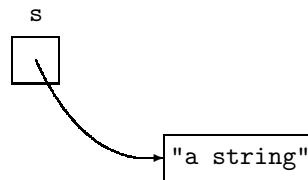
As well as assigning constants to string variables, we can also assign one string variable to another. The result is similar in some ways to that seen with assignments involving variables of primitive types: the value stored in one variable is copied into the other variable. Now, however, the copying involves references and so the effect is quite different, as shown in the next example.

Example 4

Suppose that we have created a string object by writing

```
String s = "a string";
```

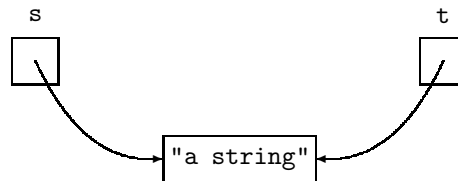
to produce the following situation:



If we now write

```
String t = s;
```

the value in `s` will be copied into `t`. Since `s` and `t` now contain the same value and they are both reference variables, they both refer to the same string.



The `print` and `println` methods that we have been using to print string constants can also be used to print string variables. We can also use the concatenation operator to print any combination of string values.

Example 5

If the `String` variable `answer` has the value "Everest", then the statement

```
System.out.println("The highest peak is " + answer + ".");
```

would print

```
The highest peak is Everest.
```

Exercises 1.8

1. Draw diagrams like those shown in the text to illustrate the result of executing the following statements.

- | | |
|---|--|
| (a) <code>int a = 1;</code>
<code>String b = "2";</code> | (b) <code>String a = "first";</code>
<code>String b = "second";</code> |
| (c) <code>String a = "one";</code>
<code>String b = a;</code>
<code>a = "two";</code> | (d) <code>String a = "ein";</code>
<code>String b = "zwei";</code>
<code>a = b;</code>
<code>b = "drei";</code> |

2. What would be printed by the following fragment?

```
String c = "cat";
String d = "dog";
String s = c;
c = d;
d = s;
System.out.println("c is " + c);
System.out.println("d is " + d);
```

1.9 Printing and Reading Values of Variables

The `print` and `println` methods that we have used to print strings can also be used to print values of variables of any type.

Example 1

The following program prints the value of an `int` variable.

```
class PrintValue
{
    // This program assigns a value to the variable
    // "number" and then prints that value.
```

```
public static void main (String[] args)
{
    int number = 37;
    System.out.println(number);
}
```

Of course, we must be sure that variables have values before we attempt to print them.

Example 2

The following program will produce an error message if we attempt to compile it.

```
class NoValue
{
    // This program is wrong because the variable
    // "number" is never given a value.
    public static void main (String[] args)
    {
        int number;
        System.out.println(number);
    }
}
```

In the statement `System.out.println("Hello");` the string `"Hello"` contained within the parentheses is called the *argument* of the `println` method. The `print` and `println` methods both print strings. If we give an argument that is not a string, it must be converted to a string. For all of Java's primitive types, this conversion is done automatically.

It is also possible to use a mixture of strings and variables of other types as the argument of the `print` or `println` methods. In this case, Java will, again, automatically convert any non-string values to strings. Then the concatenation operator (+) can be used to produce one string for printing.

Example 3

If the `String` variable `friend` refers to the string "Nadine" and the `int` variable `age` has the value 17, then the statement

```
System.out.println(friend + " is " + age + " years old");
```

will print

```
Nadine is 17 years old
```

We must be careful when we print more than one variable with one `print` or `println` statement as the results may not be what we want.

Example 4

If the `byte` variables `height` and `width` have values 4 and 6, then the statement

```
System.out.println("Dimensions are " + height + width);
```

will print

```
Dimensions are 46
```

It would be better to write

```
System.out.println("Dimensions are " + height  
                  + " and " + width);
```

to print

```
Dimensions are 4 and 6
```

As the previous examples illustrate, integer values are converted to strings of just the right length before printing. To print `boolean` variables, the strings "`true`" or "`false`" are generated. For `char` variables, a one-character string is produced. For floating point variables, the values are normally converted to strings containing 8 digits for `float` values and 16 digits for `double` values. However, any trailing zeros after a decimal point will not be printed.

1.12 Review Exercises 1

1. What will this program print?

```
class JustWriting
{
    public static void main (String[] args)
    {
        System.out.print("Here we are ");
        System.out.println("(finally),\n");
        System.out.print("at the end\nof\n");
        System.out.println("\nChapter 1");
    }
}
```

2. Write a *single* Java statement that will print the following, exactly as shown.

```
The handsome stranger said,
"My name is Kim, Paul Kim."
```

3. State the type of each legal constant. If the constant is illegal, explain why.

- | | |
|----------------|---------------|
| (a) -5 | (b) 37. |
| (c) '\\' | (d) 25e2.0 |
| (e) '5' | (f) "5" |
| (g) "sample\n" | (h) 133f |
| (i) ''' | (j) 'example' |
| (k) 1350L | (l) 15,472 |

4. What is the smallest type of integer that could be used to represent each of the following values?
- | | |
|----------|-------------------|
| (a) 400 | (b) 50 000 |
| (c) -100 | (d) 3 000 000 000 |
5. Rewrite in standard decimal form
- | | |
|------------------------|-----------------------|
| (a) $8.77\text{e-}4$ | (b) $299.04\text{e}0$ |
| (c) $-0.0443\text{E}6$ | (d) $-23\text{e-}4$ |
6. Identify, with reasons, any identifiers that should not be used for Java variables.
- | | |
|---------------------------------|----------------------------|
| (a) <code>1stTime</code> | (b) <code>Perimeter</code> |
| (c) <code>valueIn\$</code> | (d) <code>beginning</code> |
| (e) <code>average height</code> | (f) <code>case</code> |
7. Which of the following conversions always requires a cast?
- | | |
|---|--|
| (a) <code>long</code> to <code>double</code> | (b) <code>byte</code> to <code>char</code> |
| (c) <code>float</code> to <code>double</code> | (d) <code>byte</code> to <code>long</code> |
| (e) <code>short</code> to <code>char</code> | (f) <code>double</code> to <code>long</code> |
8. What would be printed by each fragment?
- (a) `double x = 0.093e-1, y = -3e-3;`
`System.out.println("x: " + x + "y: " + y);`
- (b) `int i = 5, j = -77;`
`System.out.println("first is "+i+"\nsecond is "+j);`
9. The following program contains five known syntax errors. Rewrite the program with those errors corrected.

```
class BadNews
/* another mess */
{
    public static main (String[] args);
    {
        int i = 7; j = 3;
        System.out.print("Smaller is " + i);
        System.out.println("  Larger is " + j)
    }
}
```

10. Draw diagrams like those in Section 1.8 to illustrate the result of executing the following statements.
- | | |
|---|---|
| (a) <code>String s = "this";</code>
<code>String t = "that";</code> | (b) <code>String s = "this";</code>
<code>String t = s;</code>
<code>s = "that";</code> |
| (c) <code>String s = "this";</code>
<code>String t = "that";</code>
<code>s = t;</code> | (d) <code>String s = "this";</code>
<code>String t = "that";</code>
<code>t = s;</code> |
11. Write and run an interactive program that asks the user to provide a string, a character, an integer, and a floating point value. After reading the values, the program should print each one on a separate line with a suitable message for each line of output.